

UNIVERSIDAD POLITÉCNICA DE BAJA CALIFORNIA

ACTIVIDAD 1
BASE DE DATOS LUIGIS

Procedimiento almacenado TABLA_PIZERIA
Creación, inserción y consulta de cuatro tablas

DATOS ACADÉMICOS	INFORMACIÓN
Docente	Alicia del Refugio López Aguirre
Materia	BASES DE DATOS AVANZADAS
Alumno	Peyman Miyandashti
Carrera	Ingeniería en Tecnologías de la Información e Innovación Digital
Grupo	ITIID 5AFM 2026-3
Fecha	6 de noviembre de 2026
Actividad	ACTIVIDAD 1 Para el Estudiante (desarrollar en clase)

El presente reporte documenta los cuatro puntos de la actividad, presenta el código T-SQL completo para Microsoft SQL Server, cinco registros de ejemplo por tabla, resultados esperados, comprobaciones y una conclusión. Los datos son didácticos y se han diseñado para este ejercicio.

Nota técnica: los resultados que aparecen en las tablas son esperados a partir de los INSERT. No se presenta una ejecución real en el SQL Server del estudiante.



1. Introducción y objetivo

Una pizzería necesita organizar sus clientes, el catálogo de pizzas, los tamaños disponibles y las órdenes de compra. Una base de datos relacional permite mantener cada entidad en una tabla independiente y vincular los pedidos con los datos correspondientes mediante claves foráneas.

Objetivo general: crear la base de datos LUIGIS y un procedimiento almacenado llamado TABLA_PIZERIA que genere cuatro tablas - CLIENTE, PIZZA, SIZE y ORDENES - y capture cinco registros de ejemplo en cada una.

1.1. Herramienta y conceptos aplicados

Concepto	Aplicación en LUIGIS
CREATE DATABASE / USE	Crean y seleccionan la base de datos.
CREATE PROCEDURE / EXEC	Definen y ejecutan un procedimiento almacenado.
CREATE TABLE / INSERT INTO	Generan la estructura y cargan los cinco registros.
PRIMARY KEY / FOREIGN KEY	Identifican cada fila y vinculan las órdenes con catálogos.
SELECT / INNER JOIN	Muestran los datos individuales y las órdenes completas.
GO	Separa lotes en SQL Server Management Studio (SSMS).

2. Preguntas, respuestas y desarrollo

2.1. Punto 1 - Crear la base de datos LUIGIS

Pregunta: ¿Cómo crear la base de datos llamada LUIGIS?

Respuesta: Se usa CREATE DATABASE LUIGIS. Para evitar un error cuando la base ya existe, el código comprueba previamente su existencia con DB_ID.

```
IF DB_ID(N'LUIGIS') IS NULL
BEGIN
    CREATE DATABASE LUIGIS;
END;
GO
```

Resultado esperado: la base LUIGIS queda disponible en el Explorador de objetos de SSMS, dentro de Bases de datos. Si ya existía, se conserva.

2.2. Punto 2 - Poner en uso la base de datos

Pregunta: ¿Cómo seleccionar LUIGIS para ejecutar las instrucciones siguientes?

Respuesta: Con USE LUIGIS. La instrucción establece el contexto de la base para los lotes posteriores.




```
USE LUIGIS;
GO
```

Resultado esperado: las instrucciones CREATE TABLE y CREATE PROCEDURE se ejecutarán en LUIGIS y no en otra base de datos.

2.3. Punto 3 - Procedimiento almacenado TABLA_PIZERIA

Pregunta: ¿Qué hace el procedimiento TABLA_PIZERIA?

Respuesta: Al ejecutar EXEC dbo.TABLA_PIZERIA, se crean las cuatro tablas requeridas, se establecen sus claves y relaciones, y se insertan exactamente cinco registros en cada una.

Primero se elimina una versión anterior del procedimiento, si existe. Después se define de nuevo. Dentro de él, las tablas se eliminan en un orden compatible con las relaciones: ORDENES antes que las tablas a las que hace referencia. Esto permite repetir la práctica.

Importante: volver a ejecutar el procedimiento elimina los datos anteriores de estas cuatro tablas y restablece los cinco registros didácticos. No debe utilizarse sobre información real sin una copia de seguridad.

2.4. Diseño de las tablas y sus relaciones

Tabla	Columnas solicitadas	Clave
CLIENTE	cve_c, nombre, tel, dire	cve_c (PK)
PIZZA	cve_p, nombre, ingrediente	cve_p (PK)
SIZE	cve_t, nombre	cve_t (PK)
ORDENES	cve_o, cve_c, cve_p, cve_t, fecha, precio	cve_o (PK); cve_c / cve_p / cve_t (FK)

Modelo relacional: CLIENTE (1) → (N) ORDENES; PIZZA (1) → (N) ORDENES; SIZE (1) → (N) ORDENES. La tabla ORDENES concentra las referencias a las tres tablas anteriores. Se escribe [SIZE] entre corchetes para utilizar de forma explícita ese nombre de tabla en T-SQL.



3. Cinco registros de cada tabla

3.1. CLIENTE - Registro de clientes

cve_c	nombre	tel	dire
1	Ana Lopez	6861001001	Av. Reforma 101
2	Juan Perez	6861001002	Calle Morelos 202
3	Maria Garcia	6861001003	Blvd. Lazaro Cardenas 303
4	Luis Torres	6861001004	Calzada Independencia 404
5	Sofia Ramirez	6861001005	Av. Madero 505

Función: identifica al cliente mediante cve_c y conserva nombre, teléfono y dirección de entrega.

3.2. PIZZA - Catálogo de productos

cve_p	nombre	ingrediente
1	Pepperoni	Mozzarella, tomate, pepperoni
2	Hawaiana	Mozzarella, jamón, piña
3	Mexicana	Mozzarella, chorizo, jalapeño, cebolla
4	Vegetariana	Mozzarella, champiñones, pimiento, aceitunas
5	Cuatro quesos	Mozzarella, manchego, parmesano, gouda

Función: almacena cinco variedades y los ingredientes que las distinguen.

3.3. SIZE - Catálogo de tamaños

cve_t	nombre
1	Individual
2	Chica
3	Mediana
4	Grande
5	Familiar

Función: define cinco opciones de tamaño asociables a una orden.



3.4. ORDENES - Registro de pedidos

cve_o	cve_c	cve_p	cve_t	fecha	precio (MXN)
101	1	1	3	2026-11-06	189.00
102	2	2	4	2026-11-06	249.00
103	3	3	2	2026-11-06	149.00
104	4	4	5	2026-11-06	299.00
105	5	5	1	2026-11-06	109.00

Función: cada orden tiene una clave propia y relaciona cliente, pizza y tamaño. El precio se almacena en DECIMAL(10,2) para representar cantidades monetarias con dos decimales.

4. Comprobación y resultados esperados

4.1. Ejecución del procedimiento

```
EXEC dbo.TABLA_PIZERIA;
GO
```

Resultado esperado: se crean CLIENTE, PIZZA, SIZE y ORDENES dentro de LUIGIS, con cinco filas en cada tabla. SET NOCOUNT ON reduce los mensajes intermedios de filas afectadas.

4.2. Consulta de todas las tablas

```
SELECT * FROM dbo.CLIENTE ORDER BY cve_c;
SELECT * FROM dbo.PIZZA ORDER BY cve_p;
SELECT * FROM dbo.[SIZE] ORDER BY cve_t;
SELECT * FROM dbo.ORDENES ORDER BY cve_o;
```

Respuesta: SSMS mostrará cuatro cuadrículas de resultados, una por cada SELECT, cada una con cinco registros.

4.3. Consulta multitabla para interpretar cada pedido

```
SELECT o.cve_o AS Orden, c.nombre AS Cliente,
       p.nombre AS Pizza, t.nombre AS Tamano,
       o.fecha AS Fecha, o.precio AS Precio
FROM dbo.ORDENES AS o
INNER JOIN dbo.CLIENTE AS c ON o.cve_c = c.cve_c
INNER JOIN dbo.PIZZA AS p ON o.cve_p = p.cve_p
INNER JOIN dbo.[SIZE] AS t ON o.cve_t = t.cve_t
ORDER BY o.cve_o;
```



Orden	Cliente	Pizza	Tamaño	Precio (MXN)
101	Ana Lopez	Pepperoni	Mediana	189.00
102	Juan Perez	Hawaiana	Grande	249.00
103	Maria Garcia	Mexicana	Chica	149.00
104	Luis Torres	Vegetariana	Familiar	299.00
105	Sofia Ramirez	Cuatro quesos	Individual	109.00

Interpretación: INNER JOIN sustituye los códigos de las claves foráneas por nombres significativos y permite comprender quién hizo cada orden, qué pizza pidió, su tamaño y precio.

4.4. Verificación del número de registros

```
SELECT 'CLIENTE' AS Tabla, COUNT(*) AS Registros FROM dbo.CLIENTE
UNION ALL SELECT 'PIZZA', COUNT(*) FROM dbo.PIZZA
UNION ALL SELECT 'SIZE', COUNT(*) FROM dbo.[SIZE]
UNION ALL SELECT 'ORDENES', COUNT(*) FROM dbo.ORDENES;
```

Tabla	Registros esperados
CLIENTE	5
PIZZA	5
SIZE	5
ORDENES	5

Pregunta: ¿Cuál es el resultado total de la actividad?

Respuesta: Se obtienen 4 tablas y 20 filas en total, distribuidas en grupos de cinco, con claves primarias en todas las tablas y tres claves foráneas en ORDENES.

5. Instrucciones para Microsoft SQL Server Management Studio

Paso 1: abrir SSMS y conectarse al servidor local. Paso 2: hacer clic en Nueva consulta. Paso 3: copiar el contenido completo del archivo adjunto Actividad1_LUIGIS_Pizzeria.sql. Paso 4: presionar Ejecutar o F5. Paso 5: observar las cuadrículas de Resultados y actualizar Bases de datos / LUIGIS / Tablas en el Explorador de objetos.

Nota: en SSMS sí se utiliza GO porque CREATE PROCEDURE debe iniciar su propio lote. GO es un separador interpretado por SSMS, no una instrucción SQL que se almacene dentro del procedimiento.



6. Conclusión personal

Con esta actividad aprendí a organizar los datos de una pizzería en una base de datos relacional. Comprendí que no es necesario repetir el nombre del cliente, la pizza y el tamaño dentro de cada orden, ya que las claves permiten relacionar las tablas y recuperar los detalles mediante JOIN. También identifiqué la utilidad de un procedimiento almacenado para automatizar la creación de tablas y la inserción de registros de prueba.

El resultado esperado de la práctica es una base de datos LUIGIS con cuatro tablas y cinco registros en cada una. Las consultas de verificación y la unión multitabla permiten revisar la información de forma más clara. Este ejercicio me ayudó a relacionar los comandos DDL, DML y DQL en un mismo flujo de trabajo y a entender la importancia de las claves primarias y foráneas para la integridad de los datos.

Anexo A. Código SQL completo listo para Nueva consulta

El siguiente anexo reproduce el script entregado por separado en formato .sql. Los comentarios del archivo contienen las instrucciones de uso y las consultas de comprobación.




```
-- ACTIVIDAD 1 - LUIGIS / SQL SERVER / SSMS
-- Ejecutar el script completo en una ventana Nueva consulta.
-- Si la base ya existe, se reutiliza (no se eliminan otras bases).
IF DB_ID(N'LUIGIS') IS NULL
BEGIN
    CREATE DATABASE LUIGIS;
END;
GO
USE LUIGIS;
GO

-- CREATE PROCEDURE requiere comenzar un lote nuevo.
-- Se elimina el procedimiento anterior si ya fue creado.
DROP PROCEDURE IF EXISTS dbo.TABLA_PIZERIA;
GO
CREATE PROCEDURE dbo.TABLA_PIZERIA
AS
BEGIN
    SET NOCOUNT ON;

    -- Borrar tablas existentes en orden de dependencias.
    DROP TABLE IF EXISTS dbo.ORDENES;
    DROP TABLE IF EXISTS dbo.CLIENTE;
    DROP TABLE IF EXISTS dbo.PIZZA;
    DROP TABLE IF EXISTS dbo.[SIZE];

    CREATE TABLE dbo.CLIENTE (
        cve_c INT NOT NULL PRIMARY KEY,
        nombre VARCHAR(60) NOT NULL,
        tel VARCHAR(20) NOT NULL,
        dire VARCHAR(100) NOT NULL
    );

    CREATE TABLE dbo.PIZZA (
        cve_p INT NOT NULL PRIMARY KEY,
        nombre VARCHAR(60) NOT NULL,
        ingrediente VARCHAR(160) NOT NULL
    );

    CREATE TABLE dbo.[SIZE] (
        cve_t INT NOT NULL PRIMARY KEY,
        nombre VARCHAR(30) NOT NULL
    );
```




```
    CREATE TABLE dbo.ORDENES (
        cve_o INT NOT NULL PRIMARY KEY,
        cve_c INT NOT NULL,
        cve_p INT NOT NULL,
        cve_t INT NOT NULL,
        fecha DATE NOT NULL,
        precio DECIMAL(10,2) NOT NULL,
        CONSTRAINT FK_ORDENES_CLIENTE FOREIGN KEY (cve_c) REFERENCES
   dbo.CLIENTE(cve_c),
        CONSTRAINT FK_ORDENES_PIZZA FOREIGN KEY (cve_p) REFERENCES dbo.PIZZA(cve_p),
        CONSTRAINT FK_ORDENES_SIZE FOREIGN KEY (cve_t) REFERENCES dbo.[SIZE](cve_t),
        CONSTRAINT CK_ORDENES_PRECIO CHECK (precio >= 0)
    );

    INSERT INTO dbo.CLIENTE (cve_c, nombre, tel, dire) VALUES
    (1, 'Ana Lopez', '6861001001', 'Av. Reforma 101'),
    (2, 'Juan Perez', '6861001002', 'Calle Morelos 202'),
    (3, 'Maria Garcia', '6861001003', 'Blvd. Lazaro Cardenas 303'),
    (4, 'Luis Torres', '6861001004', 'Calzada Independencia 404'),
    (5, 'Sofia Ramirez', '6861001005', 'Av. Madero 505');

    INSERT INTO dbo.PIZZA (cve_p, nombre, ingrediente) VALUES
    (1, 'Pepperoni', 'Queso mozzarella, salsa de tomate, pepperoni'),
    (2, 'Hawaiana', 'Queso mozzarella, jamon, pina'),
    (3, 'Mexicana', 'Queso mozzarella, chorizo, jalapeno, cebolla'),
    (4, 'Vegetariana', 'Queso mozzarella, champinones, pimiento, aceitunas'),
    (5, 'Cuatro quesos', 'Mozzarella, manchego, parmesano, gouda');

    INSERT INTO dbo.[SIZE] (cve_t, nombre) VALUES
    (1, 'Individual'),
    (2, 'Chica'),
    (3, 'Mediana'),
    (4, 'Grande'),
    (5, 'Familiar');

    INSERT INTO dbo.ORDENES (cve_o, cve_c, cve_p, cve_t, fecha, precio) VALUES
    (101, 1, 1, 3, '2026-11-06', 189.00),
    (102, 2, 2, 4, '2026-11-06', 249.00),
    (103, 3, 3, 2, '2026-11-06', 149.00),
    (104, 4, 4, 5, '2026-11-06', 299.00),
    (105, 5, 5, 1, '2026-11-06', 109.00);
END;
```




```
GO

-- Ejecuta el procedimiento: crea las 4 tablas e inserta 5 filas en cada una.
EXEC dbo.TABLA_PIZERIA;
GO

-- Comprueba los 5 registros de cada tabla.
SELECT * FROM dbo.CLIENTE ORDER BY cve_c;
SELECT * FROM dbo.PIZZA ORDER BY cve_p;
SELECT * FROM dbo.[SIZE] ORDER BY cve_t;
SELECT * FROM dbo.ORDENES ORDER BY cve_o;

-- Consulta multitabla para mostrar las ordenes completas.
SELECT o.cve_o AS Orden, c.nombre AS Cliente, p.nombre AS Pizza,
       t.nombre AS Tamano, o.fecha AS Fecha, o.precio AS Precio
FROM dbo.ORDENES AS o
INNER JOIN dbo.CLIENTE AS c ON o.cve_c = c.cve_c
INNER JOIN dbo.PIZZA AS p ON o.cve_p = p.cve_p
INNER JOIN dbo.[SIZE] AS t ON o.cve_t = t.cve_t
ORDER BY o.cve_o;

-- Conteos de verificacion: deben ser 5 en cada tabla.
SELECT 'CLIENTE' AS Tabla, COUNT(*) AS Registros FROM dbo.CLIENTE
UNION ALL SELECT 'PIZZA', COUNT(*) FROM dbo.PIZZA
UNION ALL SELECT 'SIZE', COUNT(*) FROM dbo.[SIZE]
UNION ALL SELECT 'ORDENES', COUNT(*) FROM dbo.ORDENES;
```